

TRAINER_SPICKZETTEL

NiFi Job-Abhängigkeiten : Trainer-Spickzettel (1 Seite)

Ul: <https://localhost:8443/nifi> · Login: `admin` / siehe `nifi_admin_credentials.txt`
Start/Stop: Rechtsklick auf Process Group → **Start** / **Stop**. Einwerfen: `cp` aus `data/samples/`. **GetFile** **pollt alle 30s** : nach dem Einwerfen 30-40s Geduld (oder Polling Interval = `2 sec`).

`N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0`

Ablauf pro Lab: Starten → Einwerfen → Erwartung → Reset

Lab 1 : Datenabhängigkeit (success/failure)

- **Start:** PG "Lab 1"
- **Einwerfen:** `cp $N/data/samples/orders.csv $N/data/input/lab1/`
- **Erwartung:** 5 Zeilen in DB-Tabelle `orders`, Marker in `data/reports/`
- **Fehlerpfad zeigen:** `cp $N/data/samples/orders_broken.csv $N/data/input/lab1/` → landet in `data/output/errors/`, DB unverändert
- **Check:** `sqlite3 $N/data/db/fmea.db "SELECT COUNT(*) FROM orders;"`
- **Reset:** `sqlite3 $N/data/db/fmea.db "DELETE FROM orders;"` + `find $N/data/{reports,output/errors,input/lab1} -type f -delete`

Lab 2 : Wait/Notify (echtes "B nach A")

- **Start (Reihenfolge!):** ZUERST nur **Job B**-Prozessoren starten → FlowFiles stauen an Wait (Self-Loop wächst)
- **Dann:** **Job A** starten + `cp $N/data/samples/orders.csv $N/data/input/lab2/`
- **Erwartung:** nach ~30s gibt Wait frei → Marker-Datei in `data/reports/`; `orders=5`
- **Reset:** Cache-Signal löschen (MapCacheServer aus/an) + DB/Ordner leeren

Lab 3 : Fan-in Barriere (3 Dateien)

- **Start:** PG "Lab 3"
- **Einwerfen:** `cp $N/data/samples/{orders,customers,products}.csv $N/data/input/lab3/`
- **Erwartung:** erst nach der **3.** Datei ein Paket in `data/output/paket/` (385 Bytes = alle 3 zusammen)
- **Reset:** `find $N/data/{input/lab3,output/paket} -type f -delete`

Lab 4 : Cron + Routing

- **Start:** PG "Lab 4" (GetFile cron alle 30s)
- **Einwerfen:** `cp $N/data/samples/orders.csv $N/data/samples/sensor_ok.csv $N/data/input/lab4/`
- **Erwartung:** `orders.csv` → DB (`orders=5`), `sensor_ok.csv` → `data/output/other/`

- **Reset:** DB `orders` leeren + `find $N/data/{input/lab4,output/other} -type f -delete`

Lab F1 : Pipeline (file, A→B→C)

- **Einwerfen:** `cp $N/data/samples/orders.csv $N/data/input/labF1/`
- **Erwartung:** Datei mit Attributen (`stufe`, `verarbeitet_am`) in `data/output/stage1/`
- **Reset:** `find $N/data/{input/labF1,output/stage1} -type f -delete`

Lab F2 : Gut/Schlecht (RouteOnContent)

- **Einwerfen:** `cp $N/data/samples/orders.csv $N/data/samples/sensor_ok.csv $N/data/input/labF2/`
- **Erwartung:** `orders.csv` → `data/output/gut/`, `sensor_ok.csv` → `data/output/schlecht/`
- **Reset:** `find $N/data/{input/labF2,output/gut,output/schlecht} -type f -delete`

Lab F3 : Fan-in (file, ohne DB)

- **Einwerfen:** `cp $N/data/samples/{orders,customers,products}.csv $N/data/input/labF3/`
- **Erwartung:** Paket (385 Bytes) in `data/output/paket/` erst nach der 3. Datei
- **Reset:** `find $N/data/{input/labF3,output/paket} -type f -delete`

Lab F4 : Wait/Notify (file)

- **Start (Reihenfolge!):** ZUERST **Job B**, dann **Job A** + `cp $N/data/samples/orders.csv $N/data/input/labF4/`
- **Erwartung:** Datei in `data/archive/`, nach ~30s Freigabe-Marker in `data/output/freigegeben/`
- **Reset:** Cache-Signal löschen (MapCacheServer aus/an) + Ordner leeren

Kernbotschaften für die Teilnehmer

1. NiFi steuert keine "Jobs", sondern den **Fluss der FlowFiles in die nächste Queue**.
2. **success/failure** = einfachste Abhängigkeit (gerichtete Connection).
3. **Wait/Notify** = echtes "B nach A" über ein Signal (entkoppelte Flows).
4. **MergeContent / Wait-Counter** = Barriere ("erst wenn N da sind").
5. **Cron + RouteOnAttribute** = zeit- und bedingungsgesteuerte Auslösung.

Häufige Stolpersteine (sofort-Diagnose)

- "Passiert nichts" → meist nur **GetFile 30s-Polling**, kurz warten.
- **PutDatabaseRecord invalid** → DBCPConnectionPool leer/nicht enabled (URL + Driver Class + Driver Location).
- **Wait gibt nie frei** → Notify/Wait nutzen unterschiedliche `Release Signal Identifier` oder Cache-Client; `wait`-Relationship muss Self-Loop sein.
- **Doppelte Zeilen** → `GetFile Keep Source File = true` zieht mehrfach.
- **Signal klebt** → zwischen Wait/Notify-Läufen MapCacheServer kurz aus/an.

Logging prüfen (Kurzreferenz)

`N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0` · ausführlich: `HANDS_ON_Logging.md`

Wo: - **Bulletin** am Prozessor (rotes Quadrat, Maus drüber) → 80 % sofort klar. Global:  → **Bulletin Board**. - **Log-Dateien** in `$N/logs/`: `nifi-app.log` (Fehler/Stacktraces, zuerst!), `nifi-user.log` (Login), `nifi-bootstrap.log` (Start), `nifi-request.log` (HTTP/API). - **Data Provenance** (Rechtsklick Prozessor → View data provenance): pro FlowFile alle Events, Attribute, Inhalt (vorher/nachher), Lineage-Graph. - **Status History** (Rechtsklick → View status history): Durchsatz/Queue über Zeit.

Befehle:

```
tail -f $N/logs/nifi-app.log
grep -i error $N/logs/nifi-app.log | tail -20
grep "PutDatabaseRecord" $N/logs/nifi-app.log
```

Eigene Logs: `LogAttribute` (Attribut-Dump, `Log Body=true` für Inhalt) bzw. `LogMessage` (freie Meldung mit EL) an eine Relationship hängen. **Failure nie blind auto-terminieren** → an `LogAttribute`/`PutFile` hängen.

Log-Level: in `$N/conf/logback.xml` z.B. `<logger`

`name="org.apache.nifi.processors.standard.PutDatabaseRecord" level="DEBUG"/>` → wird nach ~30s automatisch neu geladen (kein Neustart). Logger-Name = Spalte "Logger" in der Logzeile.

Lab L (Logging & Debugging): - Erfolg+INFO: `cp $N/data/samples/orders.csv $N/data/input/labL/` → Datei in `output/labL_ok/`, Attribut-Block in `nifi-app.log` - Fehler+ERROR: `cp $N/data/samples/orders.csv $N/data/input/labL_err/` → rotes Bulletin an `PutDatabaseRecord` + ERROR-Log - Reset: `find $N/data/input/labL $N/data/input/labL_err $N/data/output/labL_ok -type f -delete`

Globaler Reset (alles)

```
sqlite3 $N/data/db/fmea.db "DELETE FROM orders; DELETE FROM fmea_entities;"
find $N/data/input $N/data/output $N/data/reports $N/data/archive -type f -delete
# MapCacheServer in der UI kurz deaktivieren/aktivieren (Wait/Notify-Signale)
```